Contour Software — Section 3 & 4

Theory MCQs • Output Prediction • HTTP • Caching • Auth • SDLC • Generic Interview

Muhammad Faizan | Trainee Software Developer | Complete Interview Prep

SECTION 3 — THEORY + CODING MCQs

3A. OOP CONCEPTS — ALL MCQs

Question	Answer
What is OOP?	Programming paradigm based on objects that contain data (fields) and behaviour (methods). 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.
What is encapsulation?	Hiding internal data using private fields and exposing via public getters/setters. Protects data integrity.
What is inheritance?	Child class inherits fields and methods from parent class using 'extends'. Promotes code reuse.
What is polymorphism?	Same method name behaves differently. Compile-time: overloading. Runtime: overriding.
What is abstraction?	Hiding implementation details, showing only essential features. Done via abstract classes or interfaces.
Abstract class vs Interface?	Abstract: can have concrete methods, constructor, one inheritance. Interface: pure contract, multiple implementation, no constructor.
Method overloading vs overriding?	Overloading: same name, different parameters, same class, compile-time. Overriding: child redefines parent method, runtime.
What is final keyword?	final variable: can't reassign. final method: can't override. final class: can't extend/inherit.
What is static keyword?	Belongs to class, not instance. Shared across all objects. Called on class directly: ClassName.method().
What is a constructor?	Special method, same name as class, no return type. Called when object is created. Used to initialize fields.
What is 'this' keyword?	Refers to current instance of the class. Used to distinguish instance variable from parameter with same name.
What is 'super' keyword?	Refers to parent class. super() calls parent constructor. super.method() calls parent method.
What is an interface?	A contract. All methods abstract by default (Java 7). A class can implement multiple interfaces. Use: interface Animal { void speak(); }
Can abstract class have constructor?	Yes. But cannot be instantiated directly. Constructor called when child class is created via super().
What is coupling?	Degree of dependency between classes. Loose coupling = good. Tight coupling = bad. Achieved via interfaces.

What is cohesion?	How focused a class is on a single responsibility. High cohesion = good. Each class does one thing well.
What is SOLID?	5 principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
What is Single Responsibility?	A class should have only one reason to change. One class = one job.
What is Dependency Injection?	Providing dependencies from outside instead of creating inside. Enables testability and loose coupling.

3B. JAVA OUTPUT PREDICTION — PRACTICE

Pre/Post Increment

```
int x = 5;
System.out.println(x++);  // OUTPUT: 5  (print first, then increment)
System.out.println(x);    // OUTPUT: 6
System.out.println(++x);  // OUTPUT: 7  (increment first, then print)
System.out.println(x);    // OUTPUT: 7
```

String Comparison

```
String a = "hello";
String b = "hello";
String c = new String("hello");

System.out.println(a == b);        // true  (same string pool)
System.out.println(a == c);        // false (different object in heap)
System.out.println(a.equals(c));   // true  (same content)
System.out.println(a.equals(b));   // true
```

Static Variable

```
class Counter {
  static int count = 0;
  Counter() { count++; }
}
Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();
System.out.println(Counter.count);  // OUTPUT: 3
```

Inheritance and Overriding

```
class Animal {
  void speak() { System.out.println("Animal speaks"); }
}
class Dog extends Animal {
  void speak() { System.out.println("Dog barks"); }
}
Animal a = new Dog();  // polymorphism
a.speak();             // OUTPUT: Dog barks  (runtime binding)
```

Try-Catch-Finally

```
try {
  System.out.println("try");
  int x = 10/0;
  System.out.println("after divide");  // SKIPPED
} catch (ArithmeticException e) {
  System.out.println("catch");
} finally {
  System.out.println("finally");       // ALWAYS runs
}
// OUTPUT: try, catch, finally
```

Array vs ArrayList

```
int[] arr = {3, 1, 4, 1, 5};
System.out.println(arr.length);    // 5 (property, no brackets)

ArrayList list = new ArrayList<>();
```

```
list.add(10); list.add(20);
System.out.println(list.size());   // 2 (method, with brackets)
System.out.println(list.get(0));   // 10
```

Interface Default Method

```
interface Greet {
  default void hello() { System.out.println("Hello from interface"); }
}
class MyClass implements Greet {}
MyClass obj = new MyClass();
obj.hello();  // OUTPUT: Hello from interface
```

3C. HTTP CONCEPTS — STATUS CODES & METHODS

HTTP Methods

- GET — fetch/read data. No body. Idempotent (same result every call)
- POST — create new resource. Has body. Not idempotent
- PUT — replace entire resource. Idempotent
- PATCH — update partial resource. Not always idempotent
- DELETE — remove resource. Idempotent
- HEAD — like GET but no body returned
- OPTIONS — check what methods are allowed

Idempotent meaning:

Calling it multiple times gives same result. GET, PUT, DELETE are idempotent. POST is not — sending twice creates two resources.

Status Codes — All

- 200 — OK, success
- 201 — Created, resource made
- 204 — No Content, success but no body
- 301 — Moved Permanently
- 304 — Not Modified, use cache
- 400 — Bad Request, invalid input
- 401 — Unauthorized, not logged in
- 403 — Forbidden, logged in but no access
- 404 — Not Found
- 405 — Method Not Allowed
- 409 — Conflict, duplicate resource
- 422 — Unprocessable, validation error
- 429 — Too Many Requests, rate limited
- 500 — Internal Server Error
- 502 — Bad Gateway
- 503 — Service Unavailable

Q: What is the difference between 401 and 403?

A: 401 Unauthorized: user is NOT authenticated — not logged in, no token, or invalid token. Server doesn't know who you are. 403 Forbidden: user IS authenticated but doesn't have permission for this resource. Server knows who you are but says no.

Q: What is the difference between PUT and PATCH?

A: PUT replaces the ENTIRE resource — if you send partial data, missing fields become null. PATCH updates only the fields you send — other fields stay unchanged. PATCH is more efficient for partial updates.

3D. AUTHENTICATION vs AUTHORIZATION

	Authentication	Authorization
Question	WHO are you?	WHAT can you do?
Purpose	Verify identity	Check permissions
When	First step — login	After authentication
Example	Username + password login	Landlord can delete, tenant cannot
Mechanism	JWT token, session, OAuth	Roles, permissions, ACL
Failure code	401 Unauthorized	403 Forbidden

One liner: "Authentication proves identity, authorization determines access."

JWT Flow (Authentication):

- 1. User POSTs credentials to /login
- 2. Server validates, returns JWT token
- 3. Client sends token in every request: Authorization: Bearer
- 4. Server validates token signature — no DB lookup
- 5. Token expires — user must refresh or re-login

Role-based Authorization:

- After auth, check user role — admin, landlord, tenant
- Restrict endpoints based on role
- 403 returned if role doesn't have permission

3E. CACHING — COMPLETE

What is caching?

Storing frequently accessed data in fast memory so you don't hit the database every time. Like keeping a notepad with common answers instead of searching a library repeatedly.

Why use caching?

- Reduce database load
- Faster response time
- Save infrastructure cost
- Handle high traffic without DB bottleneck

TTL — Time To Live

How long cached data stays before it expires and refreshes from DB. Example: cache credit balance for 5 minutes TTL.

Cache Invalidation

When to clear cache. Hardest problem in CS. Strategies: TTL expiry, explicit delete on update, write-through cache.

Types of Caching

- Browser cache — CSS, JS, images stored locally. Controlled by Cache-Control headers
- Server cache — Redis/Memcached stores DB query results in memory
- CDN cache — static assets served from geographically nearest server
- Application cache — in-memory cache within the app itself
- Database cache — query result cache inside the DB engine

Redis — most common tool

- In-memory key-value store
- Sub-millisecond response time
- Supports TTL per key
- Used for: sessions, rate limiting, caching, queues

```
# Redis example
SET user:123:balance 5000 EX 300
# Key: user:123:balance
# Value: 5000
# EX 300: expires in 300 seconds
```

Q: When should you NOT cache?

A: Don't cache data that changes frequently and must be real-time accurate — like bank transaction balance, stock prices, live inventory. Cache is suitable for data that can be slightly stale like user profiles, product listings, configuration settings.

Q: What is cache miss vs cache hit?

A: Cache hit: data found in cache — returned immediately, fast. Cache miss: data NOT in cache — must query database, slower, then store result in cache for next time. High hit rate means caching is effective.

3F. SDLC — SOFTWARE DEVELOPMENT LIFECYCLE

Phase	What happens	Output
Requirements	Gather what system must do	BRS, SRS documents
System Design	How to build it — architecture, DB schema, APIs	Design documents, UML diagrams
Implementation	Write the actual code	Working code

Testing	Verify it works correctly	Test reports, bug list
Deployment	Release to production	Live system
Maintenance	Fix bugs, add features	Updated system

Model	How it works	When to use
Waterfall	Sequential, no going back, each phase complete before next	Clear fixed requirements, short projects
Agile	Iterative sprints, frequent releases, flexible	Changing requirements, long projects
Scrum	Agile framework: sprints, backlog, standups, retrospective	Team-based product development
V-Model	Each dev phase has corresponding test phase	Safety-critical systems, embedded
Spiral	Risk-driven, iterative, each loop adds features	Large high-risk projects
Prototype	Build mockup first, iterate based on feedback	When requirements unclear

Q: What is Agile?
A: Iterative software development approach. Work in short sprints (1-4 weeks). Deliver working software frequently. Adapt to changing requirements. Values: individuals over processes, working software over documentation, customer collaboration, responding to change.

Q: What is a sprint?
A: Fixed time-box in Scrum (usually 2 weeks). Team picks items from backlog, commits to delivering them by sprint end. Daily standup: what did I do, what will I do, any blockers.

Q: What is the difference between verification and validation?
A: Verification: are we building the product RIGHT? Reviews, inspections against specs — process focused. Validation: are we building the RIGHT product? Testing against actual user needs — product focused.

3G. DATA STRUCTURES & ALGORITHMS MCQs

Question	Answer
Stack vs Queue?	Stack: LIFO — Last In First Out. push/pop. Queue: FIFO — First In First Out. enqueue/dequeue.
Array vs ArrayList?	Array: fixed size, primitive types. ArrayList: dynamic size, objects only, more methods.
HashMap vs HashSet?	HashMap: key-value pairs. HashSet: unique values only, no duplicates, no keys.
LinkedList advantage?	O(1) insert/delete at known position. Array is O(n) for insert in middle.
Binary search requirement?	Array must be SORTED. Time complexity O(log n).
Best sorting algorithm?	Merge sort O(n log n) stable. Quick sort O(n log n) avg but $O(n^2)$ worst.
What is recursion?	Function calls itself. Must have base case to stop. Example: factorial, fibonacci.
What is Big O?	Describes how runtime grows with input size. O(1) constant, O(n) linear, $O(n^2)$ quadratic.
What is a heap?	Tree-based structure. Min-heap: parent <= children. Max-heap: parent >= children. Used in priority queue.
Difference between BFS and DFS?	BFS: level by level, uses queue, shortest path. DFS: goes deep first, uses stack/recursion, less memory.
What is a hash function?	Maps key to index in array. Good hash = even distribution, few collisions.
What is collision in HashMap?	Two keys produce same hash code. Java handles via chaining (linked list at bucket).
What is time complexity of ArrayList.get()?	O(1) — direct index access.
What is time complexity of LinkedList.get()?	O(n) — must traverse from head.
What is a balanced tree?	Tree where height difference between left and right subtree is at most 1. Ensures O(log n) operations.

3H. REST APIs, GIT & GENERAL MCQs

REST API MCQs

- What is REST? — Architectural style using HTTP methods for CRUD on resources identified by URLs
- What is stateless? — Server stores no session, each request has all needed info
- What is endpoint? — URL that accepts requests: /api/users
- What is JSON? — Lightweight data format for API responses: {"key": "value"}
- What is CORS? — Browser security that blocks requests from different origin unless server allows
- What is rate limiting? — Limit requests per user per time period to prevent abuse
- What is pagination? — Return data in pages instead of all at once: ?page=1&limit;=10
- What is versioning? — /api/v1/users vs /api/v2/users — backward compatibility

Git MCQs

- git clone — copy remote repo locally
- git add . — stage all changes
- git commit -m — save staged changes
- git push — upload to remote
- git pull — download + merge remote changes
- git branch — create/list branches
- git merge — combine branches
- git stash — temporarily save uncommitted work
- git reset HEAD~1 — undo last commit
- git diff — show unstaged changes

Merge vs Rebase?

- Merge: creates merge commit, preserves full history
- Rebase: rewrites commits on top of another branch, cleaner linear history
- Use merge for shared branches, rebase for local cleanup

Q: What is a pull request?

A: Request to merge your branch into another (usually main). Team reviews code before merging. Standard workflow: feature branch → push → PR → review → merge. Ensures code quality through review.

Q: What is .gitignore?

A: File listing what Git should not track. Common entries: node_modules/, .env, __pycache__/, *.log. Prevents sensitive data and unnecessary files from being committed.

3I. JAVASCRIPT MCQs

Question	Answer
var vs let vs const?	var: function-scoped, hoisted, can re-declare. let: block-scoped, not hoisted, can't re-declare. const: block-scoped, can't reassign (but object properties can change).
What is hoisting?	Variables and functions moved to top of scope before execution. var is hoisted (undefined), let/const hoisted but not initialized (temporal dead zone).
== vs === ?	== checks value only, type coercion. === checks value AND type. Always use ===. Example: '5' == 5 is true, '5' === 5 is false.
null vs undefined?	undefined: variable declared but no value assigned. null: intentionally empty value assigned by developer.
What is a Promise?	Object representing eventual completion of async operation. States: pending, fulfilled, rejected. .then() for success, .catch() for error.
What is async/await?	Syntactic sugar over Promises. async function returns Promise. await pauses execution until Promise resolves. Cleaner than .then() chains.
What is closure?	Function that remembers variables from outer scope even after outer function returns. Used for data privacy and function factories.
What is callback?	Function passed as argument to another function, called when operation completes. Can lead to callback hell if nested deeply.
What is event loop?	JavaScript is single-threaded. Event loop handles async operations: call stack, callback queue, microtask queue. Allows non-blocking I/O.

What is spread operator?	... spreads iterable into individual elements. [...arr1, ...arr2] merges arrays. {...obj1, ...obj2} merges objects.
What is destructuring?	Extract values from arrays/objects. const {name, age} = user. const [first, second] = arr.
What is typeof?	Returns string type: 'string', 'number', 'boolean', 'object', 'undefined', 'function'. typeof null === 'object' is a known bug.

JavaScript Code Output — Practice

```
// async/await example
async function getData() {
  const res = await fetch('/api/users');
  const data = await res.json();
  return data;
}

// Promise example
fetch('/api/users')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Closure example
function counter() {
  let count = 0;
  return function() { return ++count; };
}
const c = counter();
console.log(c()); // 1
console.log(c()); // 2
console.log(c()); // 3

// var hoisting
console.log(x); // undefined (not error)
var x = 5;
console.log(x); // 5
```

3J. DESIGN PATTERNS & SOFTWARE ENGINEERING

Pattern	What it is	Real example
Singleton	Only one instance of class exists	Database connection, Logger
Factory	Create objects without specifying exact class	UserFactory creates Admin or Regular user
Observer	Objects subscribe to events of another	Event listeners, pub/sub, React state
Repository	Abstraction layer between business logic and data	UserRepository.findById()
MVC	Model-View-Controller separation	Django MVT, React + API
Strategy	Define family of algorithms, swap them	Sort strategy: bubble, merge, quick
Decorator	Add behaviour to object without changing class	Python @decorator, middleware

SECTION 4 — GENERIC INTERVIEW

4A. TELL ME ABOUT YOURSELF

"I'm Faizan, CS graduate from FAST-NUCES graduating June 2026 with a CGPA of 3.30. Over the past year I've built full-stack applications and AI systems end-to-end. My strongest project is RentEase — a complete SaaS rental management system with a Django REST backend, PostgreSQL database, JWT authentication, and Next.js frontend. I also built AI-powered systems including a RAG pipeline for clinical diagnostics and fine-tuned LLMs. I learn fast, I build things, and I'm excited about Contour's Java training program and the opportunity to work on enterprise-scale systems."

4B. WHY CONTOUR?

"Contour is a subsidiary of Constellation Software — one of the top 10 software companies globally serving 100+ countries. That scale is rare. The structured 3-month training program that leads directly to full-time employment based on performance is exactly what I want as a fresh graduate. I want to learn enterprise Java development properly — not just tutorials, but real production systems with real users. Contour gives me that foundation."

4C. ALL BEHAVIORAL QUESTIONS — ANSWERS

Q: Why Java when your background is Python?

"OOP principles are the same across languages. I've studied Java formally in university — data structures, algorithms, OOP with Java. Python was my primary language for projects because it's faster to prototype AI/ML systems. But the concepts transfer directly. Give me 2 weeks and I'll be productive in Java."

Q: What are your strengths?

"I learn fast and I build things. In my final year I delivered 7+ complete projects end-to-end — full-stack, AI, automation. I don't just study concepts, I apply them immediately. I'm also comfortable working independently and taking ownership of a problem from start to finish."

Q: What is your weakness?

"I sometimes spend too much time perfecting code before moving on. I've learned to time-box tasks — set a deadline for myself, ship working code first, then optimize in a second pass. It's improved my delivery speed significantly."

Q: Tell me about a challenge you faced and how you solved it.

"In RentEase I had to implement role-based access control where landlords and tenants see completely different data. The challenge was enforcing this cleanly at the API level without spaghetti code. I created custom Django permission classes and middleware that checked the user role on every request. Separated concerns cleanly and it scales to any number of roles."

Q: Where do you see yourself in 3 years?

"After Contour's training program I want to grow into a mid-level Java developer working on enterprise systems. Contour's structure and Constellation's scale give me exactly that path. In 3 years I want to be contributing independently to production systems and mentoring newer team members."

Q: How do you handle pressure and tight deadlines?

"I prioritize tasks by impact — what breaks everything if not done vs what's nice to have. I break large tasks into smaller ones with daily targets. I also communicate early if something will be delayed — no surprises for the team. My FYP and 4 other projects in the same semester taught me to manage time under real pressure."

Q: Do you prefer working alone or in a team?

"Both, depending on the task. Deep coding work I prefer focus time alone. Architecture decisions, code reviews, debugging — I prefer collaboration. I've worked on group projects and solo projects. I can adapt."

Q: Why should we hire you?

"I'm a FAST-NUCES CS graduate who actually builds things. I have 7+ projects live on GitHub with real metrics, two internships, and I've worked across full-stack, AI, and automation. I learn fast — I taught myself RAG, fine-tuning, and cloud deployment in one semester alongside my coursework. Give me 3 months and I'll be a productive Java developer on your team."

Q: What do you know about Contour Software?

"Contour is a subsidiary of Constellation Software Inc. — one of the top 10 software companies globally, serving customers in 100+ countries. Contour started as an R&D; back-office and grew to 2,000+ staff across Karachi, Lahore, and Islamabad. The division I'm applying to is NorthStar Utilities — providing billing and customer systems to 200+ utility providers serving 3.5 million users. That's enterprise scale."

Q: What is your salary expectation?

"I'm open to Contour's standard package for Trainee Software Developers. My focus is on the training program and long-term growth here."

Q: Do you have any questions for us?

"Yes — what does the 3-month training program look like day to day? What is the typical tech stack I'll work on? And how does performance get evaluated for the full-time transition?"

LAST 15 MINS — READ THIS BEFORE TEST

✓ Authentication = WHO are you (login, JWT). Authorization = WHAT can you do (roles, permissions). 401 = not logged in. 403 = logged in but no access.

✓ Caching = fast memory store. Redis = tool. TTL = expiry time. Cache hit = found. Cache miss = not found, go to DB.

✓ GET = read. POST = create. PUT = replace all. PATCH = update part. DELETE = remove.

✓ Encapsulation = private fields + getters. Inheritance = extends. Polymorphism = overriding. Abstraction = interface/abstract.

✓ Stack = LIFO. Queue = FIFO. HashMap = key-value O(1). Binary search = sorted array O(log n).

✓ x++ = print then increment. ++x = increment then print.

✓ == checks reference. .equals() checks value. Always use .equals() for Strings.

✓ SDLC: Requirements → Design → Code → Test → Deploy → Maintain.

✓ Agile: iterative sprints, flexible. Waterfall: sequential, fixed.

✓ Why Contour: Constellation Software, top 10 globally, enterprise Java, structured training → full-time.